

4-th Place Solution

Rank	4
Team	Starry
Author	Starry
Topic ID	679222
Version	Original

Source: <https://www.kaggle.com/competitions/vesuvius-challenge-surface-detection/discussion/679222>

Retrieved with Kaggle CLI on 2026-07-07.

Thanks for host for this meaningful and interesting competition as well as detail answer for metric questions. And congratulations for winners!

This competition just show the powerful generalization of nnUNet again. It's the best segmentation framework in the world (I think) !

TLDR:

Huge nnUNet and heavy postprocessing

1 . Segmentation Model:

Model training : I did nothing just train a huge nnUNet (Resenc UNet with 7 stages covolution) with sufficient epoch (2000).

Inference: Uses nnUNetPredictor with a tile_step_size of 0.5 and test-time augmentation (mirroring) to ensure smooth and robust predictions.

Preliminary postprocessing: threshold=0.2 and remove components with size < 1000.

2 . Postprocessing: Two Stage hole Filling and Metric Hacking

• 1st Stage: PCA Hole Filling (Projection-based)

This stage targets large, obvious topological loops (Betti-1 errors) by analyzing 2D projections.

Detection: The projection_betti1_finding function projects 3D connected components onto XY, YZ, and ZX planes. It identifies "isolated regions" (background holes) that do not touch the image boundaries.

Refinement:

It extracts the 3D coordinates of these problematic surfaces and applies Principal Component Analysis (PCA) to find the optimal 2D fitting plane. The points are mapped to 2D, where a binary_fill_holes operation is performed. The filled results are re-mapped back to 3D space using the inverse_transform of the PCA. This step intend to fill some large holes.

In fact, there are another more elegant solution for this part: component detection -> PCA transformation -> RBF function fitting (torchrbf acceleration) -> interpolation -> PCA inverse transformation -> 3D dialation

However, there are a critical problem in this part:

How to separate adhering sheets ?

For those adhering sheets, this method would fail. And till now, I still can not find a good method to solve this problem. Looking forward other teams solution.

• 2nd Stage: PCA Hole Filling (Betti-Matching-based)

A more granular refinement stage using specialized topological tools.

Detection: Utilizing the `betty_matching` (C++) library, the `bmbarcode_betty1_finding` function performs a local persistent homology analysis.

Local Processing: The volume is divided into 20, 20, 20 chunks. If a local block exhibits a Betti-1 error (determined via barcode calculation), the PCA filling logic is applied locally to seal small breaks or gaps in the structure.

If one check the competition metric, it can be found that the topo score is very sensitive, even if a small hole would contribute to a large drop for topo score.

• Hacking the Metric

This specific logic is designed to optimize the final topological score by manipulating the betty-2 count. Although host claim that [practical impact at the top end should be limited](#), it still exist some bugs (I think) and can be utilized in the follow way:

Automatic Betti-2 Hole Filling: The `fill_betty2_holes` function identifies background components completely enclosed by the foreground and fills them to eliminate unwanted internal cavities. Set `betty-2 = 0`

Manual create betty2 number: In cases where specific error thresholds are met (e.g., high betty-0 or betty-1 error, for example: `betty-0 > 50` or `betty-1 > 5`, can be calculated using the `bm.compute_barcode` function), then create a small betty-2 error manually. This step is dangerous to some degree, as it can just calculate betty-0 or betty-1 numbers of the whole predictions, but in evaluation, there are ignore masks which could contribute to totally different betty-0 / 1 numbers.

It locates a voxel that is "fully enclosed" (all 26 neighbors are foreground) within the largest connected component. It sets that single voxel to 0, effectively creating a manual Betti-2 hole.

Strategy: This "hack" serves to balance the topological metrics if the evaluation criteria favor a specific distribution of Betti errors across different dimensions.

Some other thoughts but have not try:

1. end-2-end training a rbf function to get 0 holes sheets
2. traing a point cloud seperation model to seperate the adhering sheets

It must be acknowledged that my solution is not so elegant, but it do works. Looking forward to learn other teams solutions.

This writeup maybe lose some details. I will refine it in following days. Feel free to ask any questions.

Some ablations (Based on my best models)

	Public LB	Private LB
Segmentation Model + Remove dust	0.590	0.609
+ Stage1 + Stage2 + Hacking	0.602	0.622

submission notebook: <https://www.kaggle.com/code/shtljw/vcsd-4th-place-solution>

Supplemental Q&A

CPMP · 2026-02-28T01:11:26.227000

Great solution. I started looking at PP during last two days and I implemented a closest plane filling a bit similar to yours. But I didn't went as far as using binary hole filling in the plane, I rather went for filling a circle in that plane, resulting in too much filling sometimes. Moving to the plane coordinates is a great idea.

It is surprising that adding some cavities helps improve the metric. How much improvement do you get from filling cavities then adding back some?

Starry · 2026-02-28T01:20:20.297000

It can get ~0.007 (0.602 -> 0.609) improvment in Public LB, but in Private LB, it can just get ~0.001 (0.618 -> 0.619) improvement. For different stage abliton, I would update afterward :).